```cpp
#include <iostream>
#include <string>
#include <iomanip>


using namespace std;

struct Box {

    int id;
    string name;
    Box* next;

};



void insert(Box*& head);
void deleteBox(Box*& head);
void displayLeftToRight(Box* head);
void insertNewBox(Box*& head);
void displayRightToLeft(Box* headd);
int getSize(Box* head);
void moveIgnore();
void space_bar();
bool isEmpty(Box* head);
void clearList(Box*& head);



int main() {

    Box* head = nullptr;
    int number{};
    char answer{};

    cout << "How many Box's? ";
    cin >> number;


    for (int i = 0; i < number; i++) {
        cout << " Enter the data for box #" << i + 1 << ": ";
        insert(head);
    }




    cout << "Here is the data from the list\n";
```

```
50
51          displayLeftToRight(head);
52
53      cout << "Would you like to delete a box?";
54      cin >> answer;
55
56      while (toupper(answer) == 'Y') {
57          deleteBox(head);
58          cout << "Would you like to delete another?";
59          cin >> answer;
60      }
61
62
63      cout << "Here is the updated list\n";
64
65          displayRightToLeft(head);
66
67      return 0;
68
69  }
70
71
72
73
74
75  void moveIgnore() {
76      cin.ignore(numeric_limits<streamsize>::max(), '\n');
77
78  }
79
80
81
82
83
84
85  void insert(Box*& head) {
86      Box* ptr = new Box;
87
88      cout << "Enter the id: ";
89      cin >> ptr->id;
90
91      moveIgnore();
92
93
94      cout << "Enter the name: ";
95      getline(cin, ptr->name);
96
97      ptr->next = head;
98      head = ptr;
```

```
 99
100 }
101
102
103
104
105
106
107 void deleteBox(Box*& head) {
108
109     Box* lead = head;
110     Box* follow = nullptr;
111
112     if (head == nullptr) {
113         cout << "\nThe List is empty\n";
114         return;
115     }
116     int deleteId{};
117
118     cout << "Enter ID to delete: ";
119     cin >> deleteId;
120
121 //If the first box is the one to delete
122     if (lead->id == deleteId) {
123         head = lead->next;
124         delete lead;
125
126         cout << "The Box was deleted\n";
127         return;
128     }
129
130     // Search for the box with the id
131     while (lead != nullptr && lead->id != deleteId) {
132         follow = lead;
133         lead = lead->next;
134     }
135
136     //not in the list
137     if (lead == nullptr) {
138         cout << "The ID is not in the list\n";
139         return;
140     }
141
142     follow->next = lead->next;
143
144     delete lead;
145
146     cout << "Box was deleted!";
147
```

```
148
149
150 }
151
152
153 void space_bar() {
154     cout << "----------------------\n";
155
156 }
157
158
159 void displayLeftToRight(Box* head) {
160     space_bar();
161
162     cout << "----------------------\n";
163     while (head != nullptr) {
164         cout << "ID\t" << head->id << "\tName;\t" << head->name << endl;
165         head = head->next;
166     }
167 }
168
169
170
171
172
173 void insertNewBox(Box*& head) {
174     Box* ptr = new Box();
175
176     char choice{};
177
178     cout << "\nInsert at the front?";
179     cin >> choice;
180
181     if (toupper(choice) == 'Y');
182     {
183         cout << "Enter the ID: ";
184         cin >> ptr->id;
185
186         moveIgnore();
187
188         cout << "Enter Name";
189         getline(cin, ptr->name);
190
191         ptr->next = head;
192         head = ptr;
193
194         return;
195     }
196
```

```
197     cout << "\nADD at the end? ";
198     cin >> choice;
199
200     if (toupper(choice) == 'Y');
201     {
202         cout << "Enter the ID: ";
203         cin >> ptr->id;
204
205         moveIgnore();
206
207         cout << "Enter Name";
208         getline(cin, ptr->name);
209
210         ptr->next = nullptr;
211
212         if (head == nullptr) {
213
214             head = ptr;
215             return;
216         }
217
218
219
220
221         Box* temp = head;
222         /// move along
223         while (temp->next != nullptr) {
224             temp = temp->next;
225
226         }
227
228         temp->next = ptr;
229         return;
230
231
232     }
233
234     int id{};
235
236     cout << "After which id should i insert the box?";
237     cin >> id;
238
239     Box* temp = head;
240
241     while (temp != nullptr && temp->id != id) {
242         temp = temp->next;
243     }
244
245     if (temp == nullptr) {
```

```
246
247         cout << " \nID not found. Inserting at the end of the list.\n";
248
249         if (head == nullptr) {
250             head = ptr;
251         }
252
253         else {
254             temp = head;
255             while (temp->next != nullptr)
256                 temp = temp->next;
257
258                 temp->next = ptr;
259
260
261
262         }
263         return;
264
265     }
266
267     cout << "Enter the ID: ";
268     cin >> ptr->id;
269
270     moveIgnore();
271
272     cout << "Enter Name";
273     getline(cin, ptr->name);
274
275     ptr ->next = temp ->next;
276     temp->next = ptr;
277
278 }
279
280
281
282
283
284 void displayRightToLeft(Box* head) {
285
286     if (head == nullptr) {
287         return;
288     }
289     displayRightToLeft(head->next);
290     cout << "ID\t" << head->id << "\tName;\t" << head->name << endl;
291 }
292
293
294
```

```
295
296
297 int getSize(Box* head) {
298
299     if (head == nullptr)
300         return 0;
301     return 1 + getSize(head->next);
302
303 }
304
305
306
307
308
309
310
311 bool isEmpty(Box* head) {
312     return (head == nullptr);
313
314 }
315
316
317
318
319 void clearList(Box*& head) {
320
321     cout << "\n============ Clear the List ==================\n";
322
323     int count = 0;
324
325     while (head != nullptr) {
326         Box* toDelete = head;
327
328         head = head->next;
329         delete toDelete;
330
331         count++;
332
333     }
334     cout << "Freed" << count << "Boxes. List is clear\n\n";
335
336 }
```